

Plants Vs Zombies Game



Session: 2025 – 2029

Submitted by:

Saman Aslam 2025(S)-CS-78

Supervised by:

Mr. Laeeq Uz Zaman Khan Niazi

Department of Computer Science
University of Engineering and Technology
Lahore Pakistan

Table of Contents

1. Introduction	3
2. Game Storyline	4
3. WireFrames	5
Fig 1.1. Loading Screen:	5
Fig 1.2. Login Screen:	5
Fig 1.3. Main Menu Screen:	6
Fig 1.4. Levels Menu Screen:	6
Fig 1.5. Game Play (Lvl 1) Screen:	7
Fig 1.6. Game Over (win condition) Screen:	7
4. Problem Statement	8
5. Procedural Solution	8
6. Algorithm / Pseudocode	8
7. Demerits of the Solution	9
8. Object-Oriented Programming (OOP)	9
9. Design Decision (OOP Approach)	10
10. UML Diagram (Description)	10
11. Merits and Demerits of OOP Design	10
Merits:	10
Demerits:	10
Part 1: Game Framework Design and Implementation	11
1. Movement System	11
1.1 Problem Statement	11
1.2 Procedural Solution	11
1.3 Algorithm / Pseudocode	11
1.4 Demerits of the Solution	12
1.5 Object-Oriented Programming Concepts Used	12
1.6 Design Decision (OOP Approach)	12
1.7 UML Diagram	13
1.8 Merits and Demerits of OOP Design	14
1.9 Code	14
2. Collision System	14
2.1 Problem Statement	15
2.2 Procedural Solution	15
2.3 Algorithm / Pseudocode	15
2.4 Demerits	15
2.5 OOP Concepts	15
2.6 Design Decision	16
2.7 UML Diagram	16
2.8 Merits / Demerits	16

2.9 Code	16
3. Animation System	17
3.1 Problem Statement	17
3.2 Procedural Solution	17
3.3 Algorithm	17
3.4 Demerits	17
3.5 OOP	17
3.6 Design Decision	18
3.7 UML	18
3.8 Merits / Demerits	19
3.9 Code	19
4. Sound System	19
4.1 Problem Statement	19
4.2 Procedural Solution	19
4.3 Algorithm	19
4.4 Demerits	19
4.5 OOP	20
4.6 Design Decision	20
4.7 UML	20
4.8 Merits / Demerits	21
4.9 Code	21
5. GameObject & Environment Design	21
5.1 Problem Statement	21
5.2 Procedural Solution	21
5.3 Algorithm	21
5.4 Demerits	21
5.5 OOP	21
5.6 Design Decision	22
5.7 UML	22
5.8 Merits / Demerits	24
5.9 Code	24
Part 2: Game Development	24
6. Player System	24
6.1 Problem Statement	24
6.2 Procedural Solution	24
6.3 Algorithm	24
6.4 Demerits	24
6.5 OOP	25
6.6 Merits / Demerits	25
6.7 Code	25

7. Enemy System	25
7.1 Problem Statement	25
7.2 Procedural Solution	25
7.3 Algorithm	25
7.4 Demerits	26
7.5 OOP	26
7.6 Merits / Demerits	26
7.7 Code	26
8. Scoring & Health System	26
8.1 Problem Statement	26
8.2 Procedural Solution	26
8.3 Algorithm	26
8.4 Demerits	27
8.5 OOP	27
8.6 Code	27
9. File Handling System	27
9.1 Problem Statement	27
9.2 Procedural Solution	27
9.3 Algorithm	27
9.4 Demerits	27
9.5 OOP	27
9.6 Code	28
12.Conclusion	28
13.Future Work & Enhancements	28

1. Introduction

This project focuses on the design and development of a 2D desktop game inspired by Plants vs Zombies, implemented using C# and Windows Forms. The objective of the project is twofold: first, to design a reusable and extensible game framework, and second, to develop a complete playable game using the same framework.

The project emphasizes Object-Oriented Programming (OOP) principles, clean architecture, and modular design. Core game systems such as movement, animation, sound, collision handling, scoring, and game state management are implemented as independent and reusable components. The framework is designed to be flexible so that new game objects, behaviors, and levels can be added without modifying the core structure.

This assignment demonstrates practical understanding of game loops, inheritance, polymorphism, abstraction, and event-driven programming, while also focusing on creativity, user experience, and maintainable code design.

2. Game Storyline

The game is set in a peaceful neighborhood that is suddenly invaded by waves of intelligent and aggressive zombies. These zombies aim to cross the lawn and reach the player's home, destroying everything in their path. The player takes the role of a defender who must strategically place different types of plants on the lawn to stop the invasion.

Each plant possesses unique abilities such as shooting projectiles, eating zombies, producing energy (sun) or consuming zombies at close range. As the game progresses, zombies become stronger and faster, introducing new challenges that require strategic thinking and proper resource management.

The game is divided into multiple levels, where each level represents a new stage of the invasion. With every level, the number of zombies, their movement patterns, and their health increase. The player must survive all levels by preventing zombies from reaching the end of the lawn. If all waves are defeated, the player wins the game; otherwise, the game ends when zombies successfully breach the defense.

3.WireFrames

Fig 3.1. Loading Screen:



Fig 3.2. Login Screen:

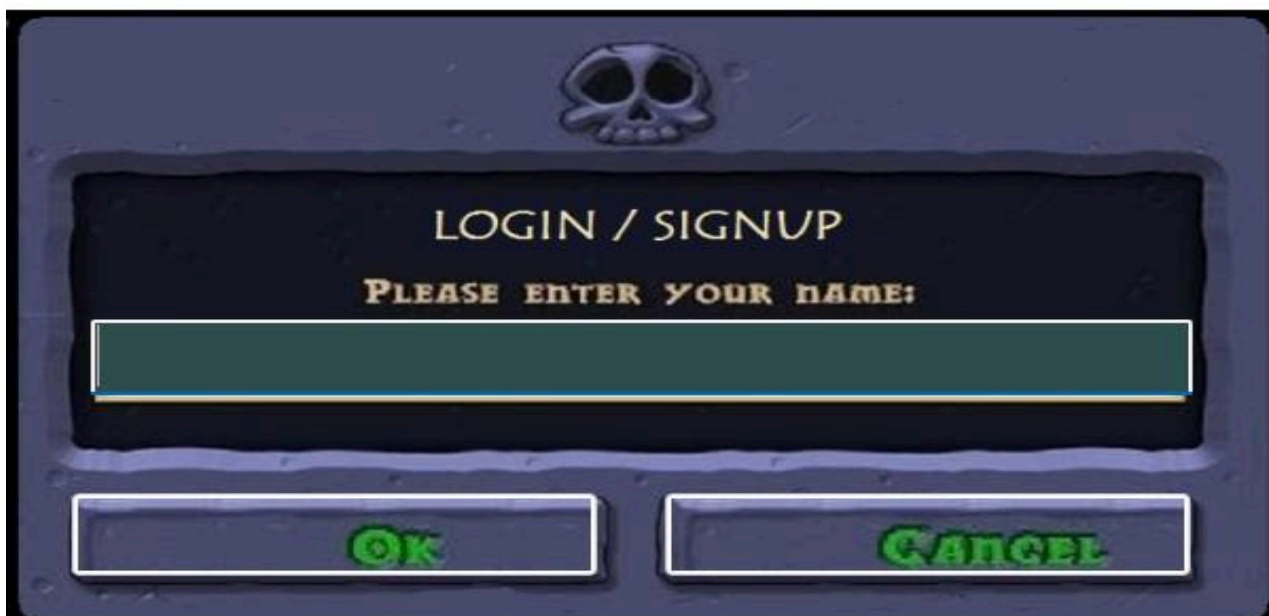


Fig 3.3. Main Menu Screen:



Fig 3.4. Levels Menu Screen:



Fig 3.5. Game Play (Lvl 1) Screen:



Fig 3.6. Game Over (win condition) Screen:



4. Problem Statement

Developing a game requires managing multiple interacting systems such as movement, animation, sound, and user interaction. Writing all logic in a single, tightly coupled structure makes the game difficult to extend, debug, and reuse.

The main problem addressed in this project is to design a **reusable game framework** that supports different movement behaviors, animations, sounds, and game objects, while also allowing the development of a complete and engaging game using the same framework.

5. Procedural Solution

To solve this problem, a modular game framework is designed using **C# and WinForms**. The framework separates responsibilities into independent components such as movement, animation, sound, and game objects. Each game object can be assigned different behaviors without changing its core structure.

A base **GameObject** class is created to represent all entities in the game. Specialized objects such as players, zombies, bullets, and environment objects inherit from this base class. Movement behaviors are implemented as independent components that can be attached to any game object. The game runs inside a continuous game loop that updates logic and redraws objects on the screen.

6. Algorithm / Pseudocode

Start Game

Initialize game framework

Load assets (images, sounds)

Create game objects and assign behaviors

While game is running

```
    Update all game objects
    Apply movement behavior
    Check collisions
    Update score and health
    Play sounds if events occur
    Draw all objects on screen
End While
```

```
If player wins
    Display victory screen
Else
    Display game over screen
End Game
```

7. Demerits of the Solution

- The modular design increases the number of classes, which can make the project structure complex.
- Improper use of inheritance may lead to tight coupling between classes.
- WinForms has limited support for advanced graphics compared to dedicated game engines.

8. Object-Oriented Programming (OOP)

The project heavily relies on Object-Oriented Programming principles:

- **Inheritance** is used to create specialized game objects from a base **GameObject** class.
- **Polymorphism** allows different game objects to respond differently to the same method calls, such as **Update()** and **Draw()**.
- **Encapsulation** ensures that object data is accessed and modified only through defined methods.
- **Abstraction** hides internal implementation details and exposes only essential behaviors.

9. Design Decision (OOP Approach)

Two approaches were considered for game object design: creating separate classes for each object type or using a generalized **GameObject** class with configurable properties. The generalized approach was selected because it allows better reusability and easier extension. New objects can be added by configuring behaviors instead of rewriting logic.

10. UML Diagram (Description)

The UML design consists of a base **GameObject** class from which **Player**, **Zombie**, **Bullet**, and **EnvironmentObject** classes inherit. Movement, animation, and sound are implemented as separate components that interact with **GameObject** through composition. This design ensures loose coupling and high flexibility.

11. Merits and Demerits of OOP Design

Merits:

- High code reusability
- Easy to extend the game with new features
- Clear separation of responsibilities
- Improved maintainability

Demerits:

- Increased design complexity

- Requires strong understanding of OOP concepts
- Debugging can be harder due to multiple layers of abstraction

Part 1: Game Framework Design and Implementation

1. Movement System

1.1 Problem Statement

In a game environment, different game objects require different movement behaviors such as moving left, moving right, jumping, or following a specific path. Implementing movement logic directly inside each game object leads to code duplication, tight coupling, and difficulty in extending the game with new behaviors.

The problem is to design a **flexible and reusable movement system** that allows different movement behaviors to be assigned to any game object dynamically without modifying its core class.

1.2 Procedural Solution

To solve this problem, a **strategy-based movement system** is implemented. An **IMovement** interface is defined that declares a common **Move()** method. Each movement behavior is implemented as a separate class that follows this interface.

Game objects do not implement movement logic themselves. Instead, they hold a reference to an **IMovement** object. During each game update cycle, the assigned movement strategy updates the position of the game object. This approach allows movement behaviors to be reused and changed at runtime.

1.3 Algorithm / Pseudocode

Start

Create a GameObject

Assign a Movement strategy to the object

While game is running

 Call Move() using the assigned movement strategy

 Update object position

End While

End

1.4 Demerits of the Solution

- Increases the number of classes in the project
- Requires careful assignment of movement strategies
- Slight overhead due to indirection through interfaces

1.5 Object-Oriented Programming Concepts Used

- **Interface:** `IMovement` defines a common movement contract
- **Polymorphism:** Different movement classes respond differently to the same `Move()` call
- **Encapsulation:** Movement logic is separated from game object logic
- **Composition:** Game objects use movement behavior instead of inheriting it

1.6 Design Decision (OOP Approach)

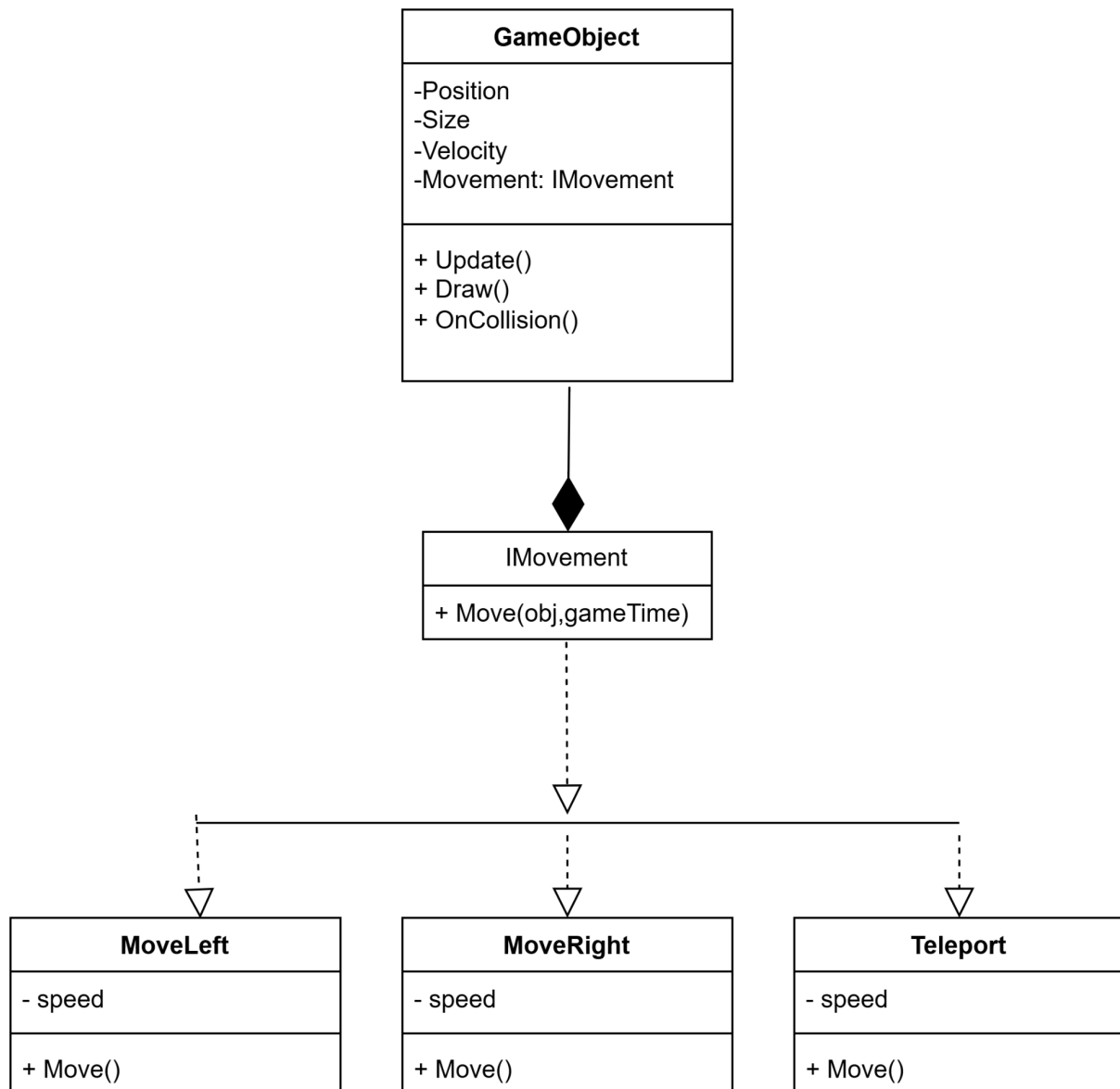
Two approaches were considered:

1. Implement movement logic directly inside each game object
2. Use a strategy-based movement system

The first approach was rejected because it causes code duplication and makes future extension difficult. The **strategy pattern** was selected because it allows movement behaviors to vary independently from game objects and supports runtime flexibility.

1.7 UML Diagram

The UML design consists of an **IMovement** interface implemented by multiple movement classes such as **MoveLeftMovement**. The **GameObject** class contains a reference to **IMovement**, showing a **has-a** relationship. This design enables loose coupling between game objects and movement logic.



1.8 Merits and Demerits of OOP Design

Merits:

- High reusability of movement logic
- Easy to add new movement behaviors
- Clean separation of responsibilities

Demerits:

- More classes to manage
- Requires understanding of interfaces and composition

1.9 Code

```
internal class MoveLeftMovement : IMovement
{
    float speed;

    public MoveLeftMovement(float speed)
    {
        this.speed = speed;
    }

    public void Move(GameObject obj , GameTime gameTime)
    {
        obj.Position = new PointF(obj.Position.X - speed, obj.Position.Y);
    }
}
```

2. Collision System

2.1 Problem Statement

In a game with multiple interactive objects, detecting and responding to collisions is essential. Without a centralized collision system, collision logic becomes scattered across classes, leading to poor maintainability and inconsistent behavior.

2.2 Procedural Solution

A centralized **CollisionSystem** is implemented to detect collisions between all collidable game objects. The system checks bounding box intersections and resolves overlaps. After detection, objects are notified through an **OnCollision()** method, allowing each object to decide its own response.

2.3 Algorithm / Pseudocode

Start

Collect all collidable objects

For each object pair

 If bounds intersect

 Resolve overlap

 Call **OnCollision** on both objects

End

End

2.4 Demerits

- Increased computational cost for many objects
- Axis-aligned bounding boxes may lack precision

2.5 OOP Concepts

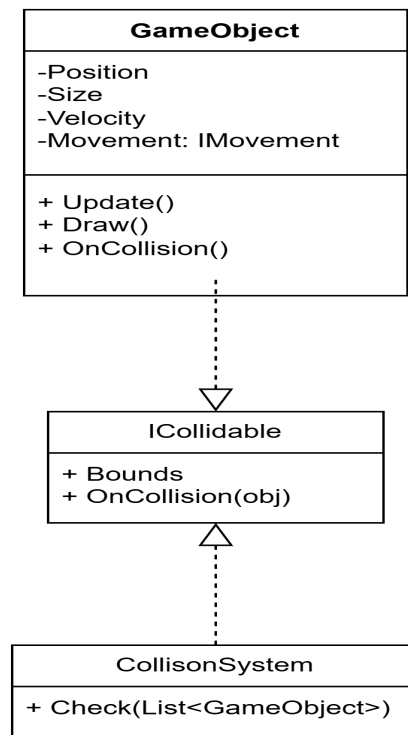
- Encapsulation: Objects handle their own collision response
- Polymorphism: Different objects react differently to collisions

2.6 Design Decision

Centralized collision handling was chosen over per-object collision logic to maintain consistency and reduce redundancy.

2.7 UML Diagram

`CollisionSystem` interacts with `ICollidable` objects. `GameObject` implements `ICollidable`.



2.8 Merits / Demerits

Merits: Clean separation, reusable logic

Demerits: Performance cost with many objects

2.9 Code

```
public class CollisionSystem
{
    public void Check(List<GameObject> objects)
    {
```

```
    // collision detection and resolution
}
}
```

3. Animation System

3.1 Problem Statement

Static images make gameplay visually dull. A flexible animation system is required to support animated characters with different states such as idle, movement, attack, and death.

3.2 Procedural Solution

An animation system using `ISprite` is implemented. `AnimatedSprite` handles frame-based animations, while `StaticSprite` is used for non-animated visuals. Game objects delegate animation updates to sprite components.

3.3 Algorithm

Start

Load animation frames

Set frame duration

On each update

 Advance frame based on delta time

Draw current frame

End

3.4 Demerits

- Memory usage for animation frames
- GIF-based animations offer limited control

3.5 OOP

- Interface (`ISprite`)

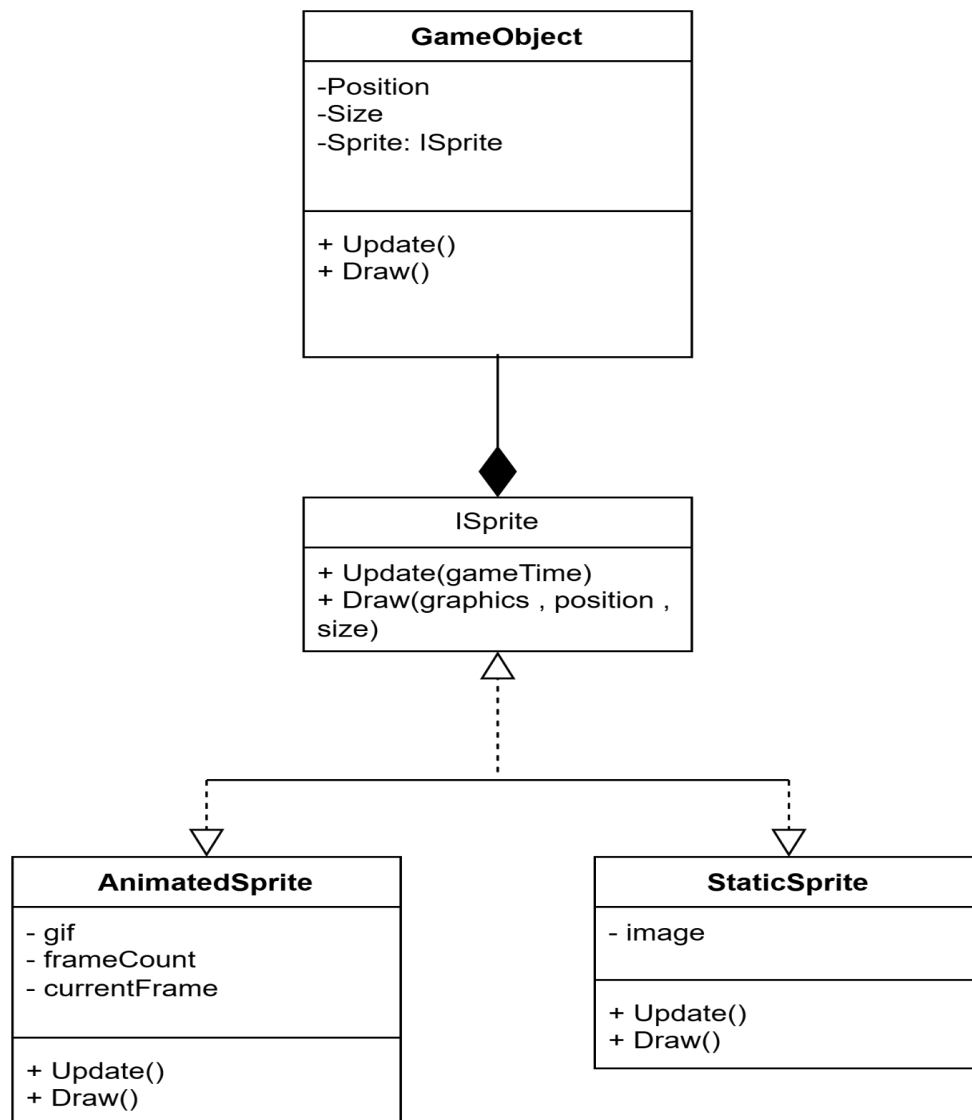
- Polymorphism (Static vs Animated sprite)
- Composition

3.6 Design Decision

Composition was preferred over inheritance to allow flexible animation swapping at runtime.

3.7 UML

GameObject has an **ISprite**. **AnimatedSprite** and **StaticSprite** implement **ISprite**.



3.8 Merits / Demerits

Merits: Reusable animations, clean design

Demerits: Requires careful asset management

3.9 Code

```
internal class AnimatedSprite : ISprite
{
    public void Update(GameTime gameTime) { }
    public void Draw(Graphics g, PointF position, SizeF size) { }
}
```

4. Sound System

4.1 Problem Statement

Games require sound feedback for immersion. Managing sounds individually inside game logic causes duplication and poor control.

4.2 Procedural Solution

A **SoundManager** class is created to manage background music and sound effects. Sounds are triggered by game events such as collisions and scoring.

4.3 Algorithm

Start

Load sound

If background

 Play looping

Else

 Play once

End

4.4 Demerits

- Limited sound control using SoundPlayer
- No advanced audio mixing

4.5 OOP

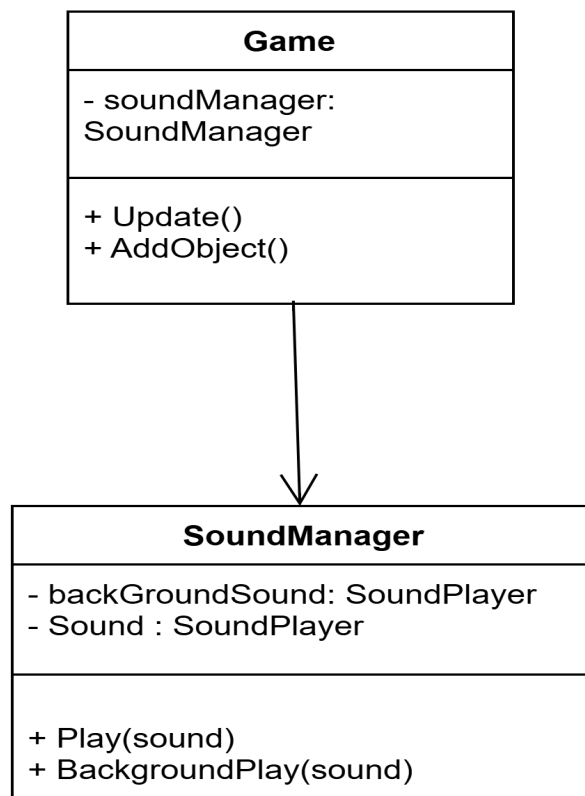
- Encapsulation
- Single Responsibility Principle

4.6 Design Decision

Central sound management ensures consistency and reuse.

4.7 UML

SoundManager is used by Game.



4.8 Merits / Demerits

Merits: Centralized control

Demerits: Basic audio features

4.9 Code

internal class SoundManager

```
{  
    public void BackgroundPlay(UnmanagedMemoryStream sound) { }  
    public void Play(UnmanagedMemoryStream sound) { }  
}
```

5. GameObject & Environment Design

5.1 Problem Statement

Different game entities require shared behavior such as drawing, updating, and collision handling.

5.2 Procedural Solution

A base `GameObject` class is designed. Static environment elements are handled through `EnvironmentObject`, allowing configuration-based reuse.

5.3 Algorithm

Create `GameObject`

Assign size, position, sprite

Update and draw every frame

5.4 Demerits

- Large base class
- Requires careful extension

5.5 OOP

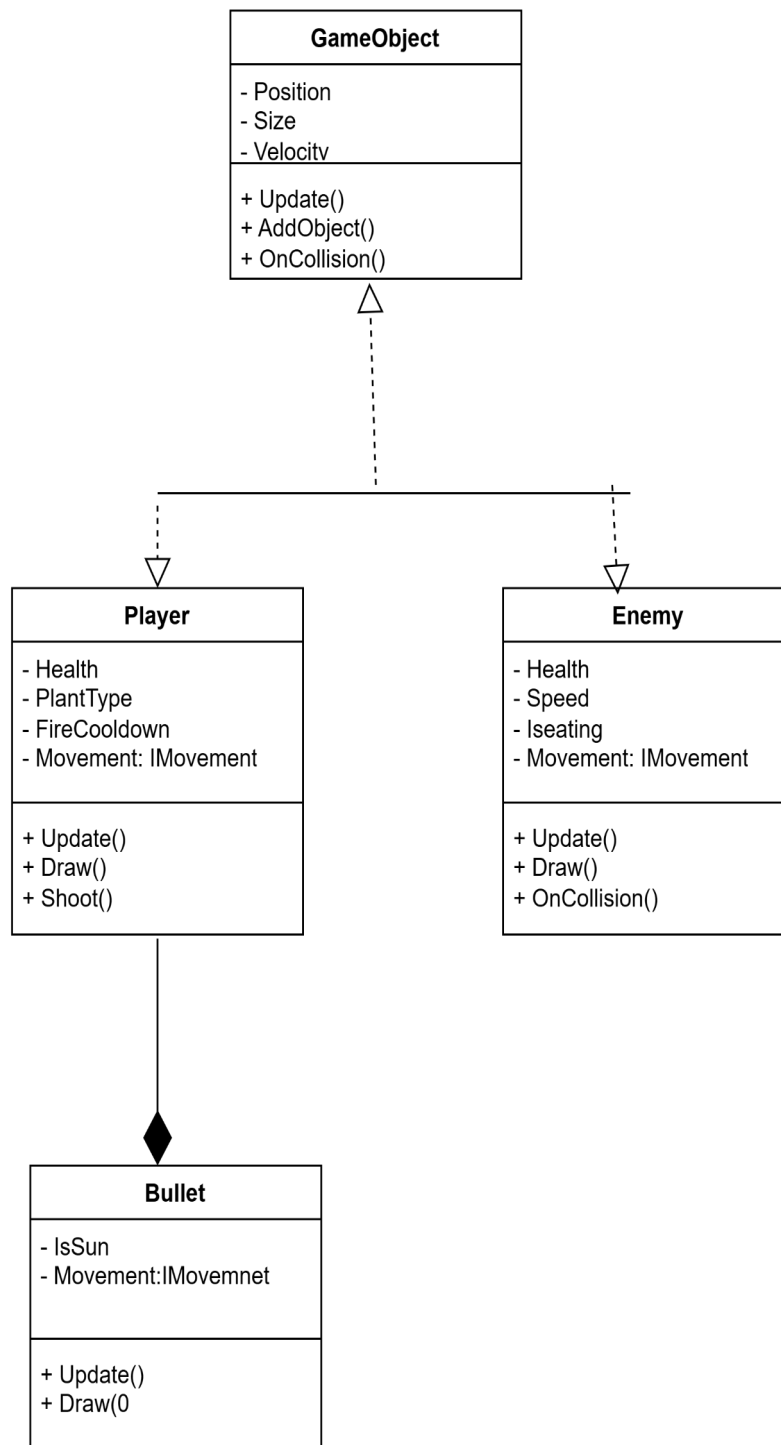
- Inheritance
- Polymorphism
- Abstraction

5.6 Design Decision

A generalized base class was chosen over many specialized classes.

5.7 UML

Player, Enemy, Bullet inherit from GameObject.



5.8 Merits / Demerits

Merits: Reusability, extensibility

Demerits: Risk of tight coupling

5.9 Code

```
public class GameObject
{
    public virtual void Update(GameTime gameTime) { }
    public virtual void Draw(Graphics g) { }
}
```

Part 2: Game Development

6. Player System

6.1 Problem Statement

The player requires health, attack, and interaction logic.

6.2 Procedural Solution

The **Player** class extends **GameObject**, handling shooting, eating, health, and collision reactions.

6.3 Algorithm

Update player

If enemy in range

 Attack

If health <= 0

 Deactivate

6.4 Demerits

- Conditional logic based on plant type

6.5 OOP

- Inheritance
- Polymorphism
- Strategy Pattern

6.6 Merits / Demerits

Merits: Flexible behaviors

Demerits: Requires state management

6.7 Code

```
public class Player : GameObject
{
    public override void Update(GameTime gameTime) { }
}
```

7. Enemy System

7.1 Problem Statement

Enemies must move, attack, and react dynamically.

7.2 Procedural Solution

The **Enemy** class handles movement, eating behavior, and death logic.

7.3 Algorithm

Move enemy

If collision with player

Attack

If health ≤ 0

Destroy

7.4 Demerits

- Requires state switching

7.5 OOP

- Inheritance
- Polymorphism

7.6 Merits / Demerits

Merits: Extendable enemy types

Demerits: State complexity

7.7 Code

```
public class Enemy : GameObject
{
    public override void Update(GameTime gameTime) { }
}
```

8. Scoring & Health System

8.1 Problem Statement

The game requires feedback for success and failure.

8.2 Procedural Solution

Scores are updated on kills and losses. Health bars reflect object health.

8.3 Algorithm

If enemy killed

Increase score

If player dies

End game

8.4 Demerits

- Manual tuning required

8.5 OOP

- Encapsulation

8.6 Code

```
int PlayerScore;
```

```
int EnemyScore;
```

9. File Handling System

9.1 Problem Statement

Game progress must persist between sessions.

9.2 Procedural Solution

File handling is used to store scores and progress.

9.3 Algorithm

Open file

Write data

Close file

9.4 Demerits

- Plain text storage

9.5 OOP

- Separation of concerns

9.6 Code

```
File.WriteAllText("save.txt", score.ToString());
```

12.Conclusion

This project demonstrates the development of a 2D Plants vs Zombies game using a reusable C# WinForms framework. It showcases **OOP principles** like composition, inheritance, and encapsulation, and implements **movement, collision, animation, and sound systems** effectively. The game achieves playable levels, scoring, health management, and visual/audio feedback while maintaining a **flexible and extensible design** for future enhancements.

13.Future Work & Enhancements

Potential improvements include:

- Smarter zombie AI and new plant types
- Dynamic or more challenging levels
- Better animations, visual effects, and audio variety
- Extended file handling for high scores and settings

These would make the game more engaging while fully utilizing the modular framework.

Student Reg. No. : 2025(S)-CS-78

Student Name. Saman Aslam

	A-Extensive Evidence	B-Convincing Evidence	C-Limited Evidence	D-No Evidence
Documentation Formatting Grade:				
Documentation Formatting Criteria: In Binder, Title Page, Header-Footers, Font Style, Font Size all are all consistence and according to given guidelines . Project Poster is professionally design and well presented				
Documentation Contents Grade:				
Documentation Contents Criteria: Title Page - Table of Contents - Project Abstract - Functional Requirements - Wire Frames – Data Flow Diagram- Data Structure (Arrays)- Function Headers and Description - Algorithms and Flow Charts of all functions- Test Cases are defined -Project Code . - Weakness in the Project and Future Directions. - Conclusion and What your Learn from the Project and Course and What is your Future Planning.				
Project Complexity Grade:				
Code Style Grade:				
Code Style Criteria: Consistent code style. Code is well indented. Variable and Function names are well defined. White Spaces are well used. Comments are added.				
Code Documentation Mapping Grade:				
Data Structure (Arrays) Grade:				
Sorting Features Grade:				
Modularity Grade:				
Modularity criteria: Functions are defined for each major feature. Functions are independent (identify from parameter list and return types)- Demo Data Functionality Added-At least Two Unit Tests are defined.				
Validations Grade:				

Recommendation Feature				
Presentation and Demo Grade:				
Student Understanding with the Code. Grade:				
Checked by:	Self Check			

Student Reg. No. :2025(S) - CS-78

Student Name. Saman Aslam

	A-Extensive Evidence	B-Convincing Evidence	C-Limited Evidence	D-No Evidence
Documentation Formatting Grade:	All the documentation meets all the criteria.	Documentation is well formatted but some of the criteria is not fulfilled.	Documentation is required a lot of improvement.	Documentation is Available
Documentation Formatting Criteria: In Binder, Title Page, Header-Footers, Font Style, Font Size all are all consistence and according to given guidelines . Project Poster is professionally design and well presented				
Documentation Contents Grade:	Documentation includes all of the criteria.	Documentation meet more than 80% of the criteria given.	Documentation meet more than 50% of the criteria.	When the documentation meet less than 50% of the criteria
Documentation Contents Criteria: Title Page - Table of Contents - Project Abstract - Function Requirements - Wire Frames -Data Flow Diagram-Data Structure (Arrays)-Function Headers and Descriptions- Algorithms and Flow Charts of all functions- Test Cases are defined -Project Code . - Weakness in the Project - Future Directions . - Conclusion and What your Learn from the Project and Course and What is your Future Planning .				
Project Complexity Grade:	Project has at least 2 user's types and each user has at least 5 functionalities.	Project complexity meet 80% criteria given in extensive evidence	Project complexity meet 50% criteria given in extensive evidence	Project complexity meet less than 50% criteria given in extensive evidence
Code Style Grade:	All Code style criteria is followed	All code style criteria followed but some	lot of improvements required in coding style.	Did not follow code style,

	A-Extensive Evidence	B-Convincing Evidence	C-Limited Evidence	D-No Evidence
Documentation on Formatting Grade:	All the documentation meets all the criteria.	Documentation is well formatted but some of the criteria is not fulfilled.	Documentation is required a lot of improvement.	Documentation is Available
		improvements required		
Code Style Criteria: Consistent code style. Code is well indented. Variable and Function names are well defined. White Spaces are well used. Comments are added.				
Code Documentation on Mapping Grade:	Code and documentation is synchronized.	Code and documentation does not synchronized at some places	Code and documentation does not synchronized at many places	Code and documentation does not synchronize
Data Structure (Arrays) Grade:	Data structure is sufficient for the project requirements	Data Structure is sufficient but require improvement to meet project requirements.	Data structure is not sufficient and need a lot of improvement	Data Structure is not properly identified and declared.
Sorting Features Grade:	Sort working 100% and generating useful report	Sorting Feature is working but sorted data is not useful for project.	Sorting feature is partial implemented	Project do not contain sorting
Modularity Grade:	Meet all Modularity criteria	Meet all Modularity criteria but at some places it is missing	Do not sufficiently meet the modularity criteria.	No modularity or very minimum modular
Modularity criteria: Functions are defined for each major feature. Functions are independent (identify from parameter list and return types)- Demo Data Functionality Added-At least Two Unit Tests are defined.				
Validations Grade:	Validations on all number type inputs are applied	Validations are applied but at some places it is missing.	Validations are missing at lot of places	No Validations are
Recommendation Feature	Proper meaning full recommendation is present into system	Partial Recommendation is implemented	Implemented but not meaning full.	Not implemented
Presentation and Demo Grade:	Presentation and Demo was 100% working	Presentation and Demo require some improvements	Presentation and Demo require a lot of improvements	Presentation was not and Demo was not working
Student Understanding with the Code. Grade:	Student has complete understanding how the code is working and knows the concept.	Student has good understand but some place he does not know the concepts	Student has a very little understand and lack the major concepts.	Student does not have any level of understanding of the code.
Checked by:	Click or tap here to enter text.			